

Übung 10: Dynamische Programmierung I und II

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie können ...

1. Auf einer Zugstrecke gibt es N Bahnhöfe, die nacheinander $(0, 1, 2, \dots, N-1)$ angefahren werden. Sie möchten den Zug von Bahnhof 0 nach Bahnhof $N-1$ nehmen. Die Kosten einer Fahrkarte von Bahnhof i zu Bahnhof j sind für alle Bahnhofspaare (i, j) gegeben, falls j größer ist als i . Die Frage ist von welchem zu welchem Bahnhof Sie jeweils ein Ticket kaufen sollten, um zwischen allen Bahnhöfen ein gültiges Ticket zu haben und dabei für Ihre Gesamtstrecke (von Bahnhof 0 nach Bahnhof $N-1$) insgesamt minimale Fahrtkosten haben. Der folgende Code bestimmt die minimalen Fahrtkosten, um den Zielbahnhof $N-1$ zu erreichen. Wenn Sie die `main` Funktion ausführen, werden für beispielhafte Ticketkosten mit $N = 4$ die minimalen Fahrtkosten berechnet. Die Fortsetzung der Frage befindet sich auf der nächsten Seite.

```
// A naive recursive solution to find min cost path from station 0 to station N-1
class shortest_path
{
    static int INF = Integer.MAX_VALUE, N = 4;
    // A recursive function to find the shortest path from source 's' to destination 'd'.
    static int minCostRec(int cost[][], int s, int d)
    {
        // If source is same as destination or destination is next to source
        if (s == d || s+1 == d)
            return cost[s][d];

        // Initialize min cost as direct ticket from source 's' to destination 'd'.
        int min = cost[s][d];

        // Try every intermediate vertex to find minimum
        for (int i = s+1; i<d; i++)
        {
            int c = minCostRec(cost, s, i) + minCostRec(cost, i, d);
            if (c < min)
                min = c;
        }
        return min;
    }

    // This function returns the smallest possible cost to
    // reach station N-1 from station 0. This function mainly uses minCostRec().
    static int minCost(int cost[][])
    {
        return minCostRec(cost, 0, N-1);
    }

    public static void main(String args[])
    {
        int cost[][] = { {0, 15, 80, 90},
                        {INF, 0, 40, 50},
                        {INF, INF, 0, 70},
                        {INF, INF, INF, 0}
                      };
        System.out.println("The Minimum cost to reach station "+ N + " is " + minCost(cost));
    }
}
```

Zeigen Sie, dass dieser Code ineffizient ist, da die Funktion `minCostRec` häufig mit identischen Argumenten aufgerufen wird. Zeichnen Sie dazu den Rekursionsbaum der Methode `minCostRec` für das Beispiel $N = 4$. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `s` und `d` einzuzichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



2. Gegeben eine positive ganze Zahl n , zählen Sie alle n -stelligen Binärzahlen, die keine direkt aufeinanderfolgende Zahlen 1 besitzen.

Zum Beispiel lauten für $n = 5$, die Binärzahlen, die die gegebene Eigenschaft erfüllen:

[00000, 00001, 00010, 00100, 00101, 01000, 01001, 01010, 10000, 10001, 10010, 10100, 10101].

Die Brute Force Lösung lautet wie folgt:

```
# zählt alle n-stelligen Binärzahlen ohne aufeinanderfolgende 1er
def countStrings(n, lastDigit=0):
    # Basisfall
    if n == 0:
        return 0

    #, wenn nur noch eine Ziffer übrig ist
    if n == 1:
        return 1 if (lastDigit == 1) else 2

    # Wenn die letzte Ziffer 0 ist, können wir an der aktuellen Position sowohl 0 als auch 1 haben
    if lastDigit == 0:
        return countStrings(n - 1, 0) + countStrings(n - 1, 1)
    # Wenn die letzte Ziffer 1 ist, können wir an der aktuellen Position nur 0 haben
    else:
        return countStrings(n - 1, 0)

if __name__ == '__main__':
    # Gesamtzahl der Ziffern
    n = 5

    print(f'The total number of {n}-digit binary numbers without any consecutive 1\'s'
          f' is {countStrings(n)}')
```

- Zeichnen Sie den sich ergebenden Rekursionsbaum für $n = 5$. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `n` und `lastDigit` einzuzichnen.
- Schreiben Sie unter jedes Blatt des Rekursionsbaums die Binärzahlen, die durch dieses Blatt repräsentiert werden.
- Passen Sie den Code an, indem Sie Memoization nutzen.

3. Sie steigen eine Treppe hinauf. Die Treppe hat n Stufen. Jedes Mal können Sie entweder 1 oder 2 Stufen nach oben steigen. Auf wie viele verschiedene Arten (mit wie viel unterschiedlichen Schrittkombinationen) können Sie nach oben steigen?

Beispiel: Für $n = 3$ lautet die Antwort 3, da es drei unterschiedliche Arten gibt die Treppe hochzu-
steigen: $[1, 1, 1]$, $[1, 2]$, $[2, 1]$.

Hinweis: Dieses Problem hat eine hohe Ähnlichkeit mit der Berechnung der Fibonacci Zahlen.

Die Brute Force Lösung lautet:

```
def climbingStairs(i, n):  
    if i > n:  
        return 0  
    if i == n:  
        return 1  
    return climbingStairs(i + 1, n) + climbingStairs(i + 2, n)  
  
if __name__ == '__main__':  
    n = 4  
    print(climbingStairs(0, n))
```

- (a) Zeichnen Sie den Rekursionsbaum für $n = 4$.
(b) Machen Sie den Code effizienter, indem Sie Memoization nutzen.
(c) Entwickeln Sie eine effiziente Lösung mit dem Bottom-Up Ansatz.
4. Sie sind für die Planung eines einmonatigen Sprints in einem Projekt verantwortlich. Während dieses Sprints stehen begrenzte Ressourcen zur Verfügung, und Sie haben eine Liste von Aufgaben, die alle die gleiche Dauer (einen Monat) haben, aber unterschiedliche Ressourcenanforderungen aufweisen und unterschiedliche Priorität haben.

Ihr Ziel ist es, den Sprint so zu planen, dass Sie solche Aufgaben auswählen, die in Summe die Prioritätsanforderungen maximieren, ohne die Gesamtkapazität zu überschreiten.

Stellen Sie sich vor, Sie haben eine Liste von 10 Aufgaben, und für jede Aufgabe ist bekannt, wie viele Ressourcen sie benötigt (bspw. Anzahl Mitarbeiter) und welche Priorität sie hat (bspw. ein Wert zwischen 1 und 10, 10 entspricht max. Priorität). Ihre Aufgabe ist es, einen Algorithmus in Pseudocode als rekursiven Brute Force Ansatz zu skizzieren, um die optimale Auswahl von Aufgaben zu bestimmen, um die Prioritätsanforderungen im Sprint zu maximieren.

5. Das Problem der längsten gemeinsamen Teilsequenz (longest common subsequence - LCS) besteht darin, die längste Teilsequenz zu finden, die in gegebenen zwei Sequenzen in derselben Reihenfolge vorhanden ist, d.h. die längste Sequenz zu finden, die aus der ersten Originalsequenz durch Löschen einiger Elemente und aus der zweiten Originalsequenz durch Löschen einiger Elemente erhalten werden kann.

Beispiel: Gegeben die beiden Sequenzen

X: CBAD

Y: CAB

Die Länge der LCS beträgt 2, da die LCS lautet: CA.

Die Brute-Force Lösung lautet:

```
# Funktion zum Ermitteln der Länge der längsten gemeinsamen Teilfolge von
# Sequenzen 'X[0...m-1]' und 'Y[0...n-1]'
def LCSLength(X, Y, m, n):

    # kehrt zurück, wenn das Ende einer der beiden Sequenzen erreicht ist
    if m == 0 or n == 0:
        return 0

    #, wenn das letzte Zeichen von 'X' und 'Y' übereinstimmt
    if X[m - 1] == Y[n - 1]:
        return LCSLength(X, Y, m - 1, n - 1) + 1

    # andernfalls, wenn das letzte Zeichen von 'X' und 'Y' nicht übereinstimmt
    return max(LCSLength(X, Y, m, n - 1), LCSLength(X, Y, m - 1, n))

if __name__ == '__main__':

    X = 'CBAD'
    Y = 'CAB'

    print('The length of the LCS is', LCSLength(X, Y, len(X), len(Y)))
```

Zeigen Sie, dass dieser Code ineffizient ist, da die Funktion `LCSLength` häufig mit identischen Argumenten aufgerufen wird. Zeichnen Sie dazu den Rekursionsbaum der Methode `LCSLength` für das im Code aufgerufene Beispiel bis zur **maximalen Höhe von 3**. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `m` und `n` einzuzichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



6. Die Funktion `fib(n)` berechne die nte Fibonaccizahl. Diese sei implementiert mit einer rekursiven Brute-Force Lösung (s. Code). Die Funktion `fib_memo(n)` berechne ebenfalls die nte Fibonaccizahl, wobei bei dieser Funktion die rekursive Brute-Force Lösung durch die Memoization Technik verändert wurde. Wie viele Funktionsaufrufe macht `fib(5)` und wie viele `fib_memo(5)`? Sie müssen Ihre Antwort nicht begründen. Es dürfte hilfreich sein den Rekursionsbaum für `fib(5)` zu zeichnen.

```
public static int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

7. Das Partitionsproblem besteht darin, zu bestimmen, ob eine gegebene Menge von Ganzzahlen in zwei Teilmengen aufgeteilt werden kann, so dass die Summe der Elemente in beiden Teilmengen gleich ist. Bspw. kann die Liste [4, 2, 2] in [4] und [2, 2] partitioniert werden, da die Summe der Elemente in beiden Listen 4 beträgt.

Die Brute-Force Lösung lautet:

```
def isSubsetSum(arr, n, sum):
    if sum == 0:
        return True
    if n == 0 and sum != 0:
        return False

    # Das letzte Element ist grösser als die gesuchte Summe:
    # es kann nicht Teil einer gültigen Teilmenge sein, ignoriere es einfach
    if arr[n-1] > sum:
        return isSubsetSum(arr, n-1, sum)

    # Zwei Möglichkeiten, die mit OR verknüpft werden:
    # 1) arr[n-1] wird NICHT genommen -> suche 'sum' in den ersten n-1 Elementen
    # 2) arr[n-1] wird genommen -> suche 'sum - arr[n-1]' in den ersten n-1 Elementen
    return isSubsetSum(arr, n-1, sum) or isSubsetSum(arr, n-1, sum-arr[n-1])

def findPartion(arr, n):
    sum = 0
    for i in range(0, n):
        sum += arr[i]

    if sum % 2 != 0:
        return False

    return isSubsetSum(arr, n, sum // 2)

if __name__ == '__main__':
    arr = [4, 2, 2]
    n = len(arr)

    if findPartion(arr, n) == True:
        print("Can be divided into two subsets of equal sum")
    else:
        print("Can not be divided into two subsets of equal sum")
```

Zeichnen Sie dazu den Rekursionsbaum der Methode `isSubsetSum` für das im Code aufgerufene Beispiel. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `n` und `sum` einzuzeichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



Rechenkästchen folgen auf der nächsten Seite.